Grundlagen der Programmierung – Übung 3

Komplette Lösung. Links der Java-Code, rechts zu jeder Zeile die Erklärung, warum man es so macht.

Vorbereitung: Imports und Klasse

Java-Code	Warum macht man das so?
`import java.util.Random;`	Wir brauchen die Klasse Random aus der Java-API, um in Aufgabe 3 eine Zufallszahl zu erzeugen. Ohne import kennt der Compiler "Random" nicht.
`import java.util.Scanner;`	Scanner liest Eingaben von der Tastatur (Aufgabe 3). Liegt ebenfalls im Paket java.util und muss importiert werden.
`public class App {`	Jedes Java-Programm besteht aus Klassen. Der Klassenname muss gleich wie der Dateiname sein (App.java). In der Abgabe heißt die Klasse App.

Aufgabe 1 – One Month Calendar

Java-Code	Warum macht man das so?
`    public static void oneMonthCalendar(int days, int firstDay) {`	public = von überall aufrufbar (auch von den Tests). static = gehört zur Klasse, kein Objekt nötig. void = gibt nichts zurück, die Methode gibt nur auf der Konsole aus. days = Anzahl Tage, firstDay = 1 (Mo) bis 7 (So).
`        int column = firstDay - 1;`	In column merken wir uns die Spalte, in der wir gerade stehen (0 = Mo ... 6 = So). Wir zählen ab 0, firstDay aber ab 1, darum -1. Beispiel: Start Mittwoch (3) -> 2 leere Spalten davor (Mo, Di).
`        for (int i = 0; i < column; i++) {`	Diese Schleife läuft genau so oft, wie es leere Felder vor dem 1. Tag gibt.
`            System.out.print("   ");`	Ein leeres Feld = 3 Leerzeichen. Warum 3? Jede Zahl wird unten mit "%2d " ausgegeben = 2 Zeichen Zahl + 1 Leerzeichen = 3 Zeichen. Sonst wären die Spalten verschoben. print (nicht println), weil wir in derselben Zeile bleiben wollen.
`        }`	Ende der Schleife für die leeren Felder.
`        for (int day = 1; day <= days; day++) {`	Jetzt alle Tage von 1 bis days durchgehen. <= weil der letzte Tag (z.B. 31) auch ausgegeben werden muss.
`            System.out.printf("%2d ", day);`	printf = formatierte Ausgabe. %2d = ganze Zahl, mindestens 2 Zeichen breit, rechtsbündig. Aus 5 wird " 5", aus 12 bleibt "12". Dahinter ein Leerzeichen (laut Angabe). So stehen alle Zahlen sauber untereinander.
`            column++;`	Nach jedem Tag rücken wir eine Spalte weiter nach rechts.

Java-Code	Warum macht man das so?
`if (column == 7) {`	7 Spalten sind voll -> der Sonntag wurde gerade ausgegeben, die Woche ist fertig.
`System.out.println();`	Zeilenumbruch: die nächste Woche beginnt in einer neuen Zeile.
`column = 0;`	Spalte zurücksetzen, die neue Zeile beginnt wieder beim Montag.
`}`	Ende der if-Abfrage.
`}`	Ende der Schleife über alle Tage.
`System.out.println();`	Laut Angabe kommt am Ende noch eine Leerzeile. Endet der Monat mitten in der Woche, beendet dieses println die letzte Zeile. Endet er am Sonntag, entsteht eine echte Leerzeile.
`}`	Ende der Methode oneMonthCalendar.

Aufgabe 2 – Pseudo Random Numbers (linearer Kongruenzgenerator)

Java-Code	Warum macht man das so?
`public static long[] lcg(long seed) {`	Rückgabetyp long[] = ein Array aus long-Zahlen (laut Angabe). Parameter seed = Startwert X0. long, weil die Zahlen sehr groß werden.
`final long m = 2147483648L;`	m = 2^31. Der größte int ist 2^31 - 1 = 2147483647, also passt m NICHT in einen int -> long. Das L am Ende macht die Zahl zu einem long-Literal (ohne L: Compile-Fehler "integer number too large"). final = Konstante, darf sich nicht ändern.
`final long a = 1103515245L;`	Faktor a. Passt zwar in einen int, aber a * x wird bis ca. 2,4 * 10^18 groß. Das würde bei int überlaufen (falsche/negative Werte). long geht bis ca. 9,2 * 10^18 -> reicht.
`final long c = 12345L;`	Inkrement c, ebenfalls long, damit die ganze Rechnung in long passiert.
`long[] numbers = new long[10];`	Neues Array mit 10 Plätzen erzeugen, weil wir genau 10 Zufallszahlen liefern sollen. Alle Plätze sind anfangs 0.
`long x = seed;`	x hält immer die aktuelle Zahl Xi. Am Anfang ist das der Seed X0.
`for (int i = 0; i < numbers.length; i++) {`	10 Durchläufe (i = 0 bis 9). numbers.length statt 10, damit man die Zahl nur an einer Stelle ändern müsste.
`x = (a * x + c) % m;`	Die Formel aus der Angabe: X(i+1) = (a * Xi + c) mod m. % ist der Modulo-Operator (Rest der Division). Dadurch liegt das Ergebnis immer zwischen 0 und m - 1. Das neue x überschreibt das alte -> die nächste Zahl wird aus der vorherigen berechnet.

Java-Code	Warum macht man das so?
`            numbers[i] = x;`	Die neue Zufallszahl an Position i ins Array speichern.
`        }`	Ende der Schleife.
`        return numbers;`	Das fertige Array an den Aufrufer zurückgeben.
`    }`	Ende der Methode lcg.

Aufgabe 3 – Guessing Game

Java-Code	Warum macht man das so?
`    public static void guessingGame(int numberToGuess) {`	void, weil nur ausgegeben wird. numberToGuess = die zu erratende Zahl (1 bis 100), wird von main übergeben.
`        Scanner scanner = new Scanner(System.in);`	Neues Scanner-Objekt, das von System.in (= Tastatur) liest.
`        boolean won = false;`	Merkt sich, ob der Spieler gewonnen hat. Brauchen wir nach der Schleife, weil die Schleife aus 2 Gründen enden kann (erraten oder 10 Versuche weg). Am Anfang hat noch niemand gewonnen -> false.
`        for (int attempt = 1; attempt <= 10; attempt++) {`	Maximal 10 Versuche -> for-Schleife von 1 bis 10. Wir zählen ab 1, weil die Ausgabe "Guess number 1" lautet.
`            System.out.print("Guess number " + attempt + ": ");`	print statt println: der Cursor bleibt in der Zeile, der Benutzer tippt direkt hinter den Doppelpunkt (wie im Beispiel). + verbindet Text und Zahl zu einem String.
`            int guess = scanner.nextInt();`	Wartet auf die Eingabe und liest die nächste ganze Zahl ein.
`            if (guess == numberToGuess) {`	Fall 1: richtig geraten. == vergleicht zwei int-Werte.
`                System.out.println("You won wisenheimer!");`	Gewinner-Text exakt wie in der Angabe.
`                won = true;`	Merken, dass gewonnen wurde, damit später nicht "You lost!" kommt.
`                break;`	Schleife sofort verlassen. Das ist die zweite Abbruchbedingung ("bei Erraten der Zahl").
`            } else if (numberToGuess < guess) {`	Fall 2: die gesuchte Zahl ist kleiner als der Tipp. Achtung auf die Richtung: der Satz beschreibt die gesuchte Zahl, nicht den Tipp.
`                System.out.println("The number AI picked is lower than your guess.");`	Hinweis: tiefer raten.
`            } else {`	Fall 3: übrig bleibt nur, dass die gesuchte Zahl größer ist.
`                System.out.println("The number AI picked is higher than your guess.");`	Hinweis: höher raten.
`            }`	Ende if / else if / else.

Java-Code	Warum macht man das so?
`        }`	Ende der Schleife. Hier kommen wir an, wenn gewonnen wurde (break) oder alle 10 Versuche aufgebraucht sind.
`        if (!won) {`	! = "nicht". Nur wenn NICHT gewonnen wurde ...
`            System.out.println("You lost! Have you ever heard of divide & conquer?");`	... kommt der Verlierer-Text. "Divide & conquer": immer die Mitte raten halbiert den Bereich -> nach max. 7 Versuchen gefunden.
`        }`	Ende if.
`    }`	Ende der Methode guessingGame.
`    public static int randomNumberBetweenOneAndHundred() {`	Liefert eine Zufallszahl zurück -> Rückgabetyp int. Keine Parameter nötig.
`        Random random = new Random();`	Neues Random-Objekt erzeugen (aus java.util). Es liefert echte Java-Zufallszahlen.
`        return random.nextInt(100) + 1;`	nextInt(100) liefert 0 bis 99 (die 100 ist ausgeschlossen!). + 1 verschiebt den Bereich auf 1 bis 100. Alternative: 1 + (int)(Math.random() * 100).
`    }`	Ende der Methode.

Aufgabe 4 – Swap Arrays

Java-Code	Warum macht man das so?
`    public static boolean swapArrays(int[] array1, int[] array2) {`	boolean, weil true (getauscht) oder false (Längen verschieden) zurückkommt. Die Arrays werden als Referenz übergeben -> Änderungen am INHALT sieht auch main.
`        if (array1.length != array2.length) {`	Zuerst prüfen: sind die Längen verschieden? != heißt "ungleich". .length ist die Anzahl der Elemente.
`            return false;`	Dann sofort abbrechen und false zurückgeben (frühes return). Der Rest der Methode wird nicht mehr ausgeführt.
`        }`	Ende if. Ab hier wissen wir sicher: beide Arrays sind gleich lang.
`        for (int i = 0; i < array1.length; i++) {`	Jede Position i von 0 bis length - 1 durchgehen. Index beginnt bei 0, deshalb < und nicht <=.
`            int temp = array1[i];`	Hilfsvariable: den alten Wert von array1[i] zwischenspeichern. Ohne temp ginge er in der nächsten Zeile verloren.
`            array1[i] = array2[i];`	array1 bekommt den Wert aus array2. Jetzt steht in beiden der gleiche Wert, der alte ist nur noch in temp.

Java-Code	Warum macht man das so?
`array2[i] = temp;`	array2 bekommt den gemerkten alten Wert von array1. Tausch fertig ("Dreieckstausch").
`}`	Ende der Schleife. Warum nicht einfach array1 = array2? Java übergibt nur KOPIEN der Referenzen. Diese zu tauschen ändert in main nichts. Den Inhalt zu ändern schon.
`return true;`	Alles getauscht -> true zurückgeben.
`}`	Ende der Methode swapArrays.

Aufgabe 5 – CamelCase

Java-Code	Warum macht man das so?
`public static String camelCase(String sentence) {`	Bekommt einen Satz und gibt den neuen CamelCase-String zurück -> Rückgabetyp String.
`char[] chars = sentence.toCharArray();`	Den String in ein Array aus einzelnen Zeichen umwandeln (so verlangt es die Angabe). So können wir jedes Zeichen einzeln anschauen.
`StringBuilder result = new StringBuilder();`	Hier bauen wir das Ergebnis zusammen. Strings sind unveränderlich, jedes += würde einen neuen String erzeugen. StringBuilder ist ein veränderbarer Puffer -> effizienter (laut Angabe erlaubt).
`boolean newWord = true;`	Merkt sich, ob der nächste Buchstabe der ERSTE eines Wortes ist. Startet mit true, weil auch der allererste Buchstabe groß werden muss.
`for (char ch : chars) {`	for-each-Schleife: ch ist nacheinander jedes Zeichen aus dem Array. Wir brauchen keinen Index, nur das Zeichen.
`if (ch == ' ') {`	Fall 1: Leerzeichen. chars vergleicht man mit == und einfachen Anführungszeichen ' '.
`newWord = true;`	Leerzeichen wird NICHT angehängt (wird entfernt), aber der nächste Buchstabe beginnt ein neues Wort.
`} else if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z')) {`	Fall 2: ein Buchstabe. chars sind intern Zahlen ('a' = 97 ... 'z' = 122, 'A' = 65 ... 'Z' = 90) und liegen hintereinander. Darum kann man mit >= und <= prüfen, ob es ein Klein- ODER (||) Großbuchstabe ist.
`if (newWord) {`	Ist es der erste Buchstabe eines Wortes?
`if (ch >= 'a' && ch <= 'z') {`	Nur wenn er klein ist, muss man ihn umwandeln.
`ch = (char) (ch - 'a' + 'A');`	klein -> groß, manuell gerechnet: ch - 'a' = Position im Alphabet (z.B. 'm' -> 12). + 'A' (65) = 77 = 'M'. Das Rechnen ergibt einen int, darum der Cast (char) zurück.

Java-Code	Warum macht man das so?
`                }`	Ende if.
`                newWord = false;`	Der Wortanfang ist erledigt, die folgenden Buchstaben gehören zum selben Wort.
`            } else {`	Sonst: Buchstabe mitten im Wort.
`                if (ch >= 'A' && ch <= 'Z') {`	Ist er groß? Laut Angabe müssen Großbuchstaben mitten im Wort klein werden (z.B. "AnY" -> "Any").
`                    ch = (char) (ch - 'A' + 'a');`	groß -> klein, genau umgekehrt: Position im Alphabet + 'a'.
`                }`	Ende if.
`            }`	Ende if / else.
`            result.append(ch);`	Den (evtl. umgewandelten) Buchstaben ans Ergebnis anhängen.
`        }`	Fall 3 (kein eigener Code nötig): alle anderen Zeichen wie . , ' ? werden einfach nicht angehängt -> herausgefiltert.
`    }`	Ende der Schleife über alle Zeichen.
`    return result.toString();`	Den StringBuilder in einen normalen String umwandeln und zurückgeben.
`}`	Ende der Methode camelCase.

Aufgabe 6 – Check Digit (Modulo 11)

Java-Code	Warum macht man das so?
`public static int checkDigit(int[] code) {`	Bekommt die Ziffern als int-Array und liefert die Prüfziffer als int zurück.
`    int sum = 0;`	Summe der Produkte. Muss bei 0 starten, weil wir dazuaddieren.
`    for (int i = 0; i < code.length; i++) {`	Normale for-Schleife (nicht for-each), weil wir den Index i für das Gewicht brauchen. Funktioniert für beliebig lange Arrays.
`        sum += code[i] * (i + 2);`	Schritt 1+2: Gewicht = Position + 2 (Index 0 -> Gewicht 2, Index 1 -> 3, ...). Ziffer mal Gewicht, und mit += zur Summe addieren.
`    }`	Ende der Schleife. Beispiel 3 9 1 5 8: 6 + 27 + 4 + 25 + 48 = 110.
`    int rest = sum % 11;`	Schritt 3: Rest der Division durch 11 (Modulo). Ergebnis liegt zwischen 0 und 10. Beispiel: 110 % 11 = 0.
`    int digit = 11 - rest;`	Schritt 4: Prüfziffer = 11 - Rest. Liegt zwischen 1 und 11. Beispiel: 11 - 0 = 11.
`    if (digit == 10) {`	Schritt 5, Ausnahme 1: 10 ist keine einzelne Ziffer ...

Java-Code	Warum macht man das so?
`            digit = 0;`	... darum wird sie auf 0 gesetzt.
`        } else if (digit == 11) {`	Ausnahme 2: 11 ist auch keine einzelne Ziffer ...
`            digit = 5;`	... darum wird sie auf 5 gesetzt. Beispiel: 11 -> 5.
`        }`	Ende if / else if.
`        return digit;`	Die fertige Prüfziffer zurückgeben.
`    }`	Ende der Methode checkDigit.

main – hier wird alles getestet

Java-Code	Warum macht man das so?
`    public static void main(String[] args) {`	Einstiegspunkt: hier startet das Programm. Muss genau so heißen (public static void main(String[] args)).
`        oneMonthCalendar(31, 3);`	Test Aufgabe 1, Konsolenbeispiel 1: 31 Tage, Start Mittwoch.
`        oneMonthCalendar(28, 1);`	Test Aufgabe 1, Konsolenbeispiel 2: 28 Tage, Start Montag.
`        long[] randoms = lcg(0);`	Test Aufgabe 2: Methode mit Seed 0 aufrufen, das zurückgegebene Array speichern.
`        for (long r : randoms) {`	Laut Angabe alle Zufallszahlen des Arrays ausgeben -> Schleife über das Array.
`            System.out.println(r);`	Jede Zahl in eine eigene Zeile.
`        }`	Ende der Schleife.
`        int[] a1 = {1, 2, 3, 4, 5};`	Test Aufgabe 4: zwei Arrays aus dem Beispiel anlegen. {...} erzeugt und befüllt ein Array direkt.
`        int[] a2 = {100, 99, 60, 44, 2};`	Zweites Array, gleich lang (5 Elemente).
`        System.out.println(swapArrays(a1, a2));`	Methode aufrufen und das Ergebnis (true) ausgeben. Danach steht in a1 {100,99,60,44,2} und in a2 {1,2,3,4,5}.
`        System.out.println(camelCase("my name isn't Alice"));`	Test Aufgabe 5, erwartet: MyNameIsntAlice
`        System.out.println(checkDigit(new int[]{3, 9, 1, 5, 8}));`	Test Aufgabe 6. new int[]{...} erzeugt das Array direkt im Aufruf. Erwartet: 5
`        guessingGame(randomNumberBetweenOneAndHundred());`	Aufgabe 3 zum Schluss, weil es auf Eingaben wartet. Der Rückgabewert der Zufallsmethode wird direkt als Parameter an guessingGame übergeben.

Java-Code	Warum macht man das so?
`    }`	Ende von main.
`}`	Ende der Klasse App.